Arquitectura de tres capas

En el desarrollo de aplicaciones backend modernas, la arquitectura de tres capas se ha consolidado como un estándar para organizar el código de manera clara, escalable y mantenible. Esta arquitectura surge de la necesidad de separar responsabilidades dentro de un sistema, evitando que la lógica de negocio, la comunicación con los clientes y el acceso a los datos queden acoplados en un único bloque de código difícil de modificar o extender.

La arquitectura de tres capas divide la aplicación en módulos lógicos diferenciados, cada uno con una función específica:

1. La capa de tráfico o presentación, responsable de recibir y gestionar las solicitudes externas (por ejemplo, peticiones HTTP).
2. La capa de servicios o lógica de negocio, donde residen las reglas y los procesos que determinan el funcionamiento de la aplicación.
3. La capa de acceso a datos, que se comunica con la base de datos u otros sistemas de almacenamiento persistente.

Este modelo no solo organiza el código, sino que también facilita la escalabilidad (al poder crecer de manera ordenada), la reutilización (cada capa puede evolucionar sin afectar a las demás) y la mantenibilidad (los cambios se concentran en la capa correspondiente).

La arquitectura de tres capas es una variante de la arquitectura en capas que establece una división clara de responsabilidades. En su forma clásica se describe como:

- Capa de presentación (UI): interfaz que interactúa con el usuario.
- Capa de negocio: lógica que procesa las reglas de la aplicación.
- Capa de datos: encargada del almacenamiento y recuperación de la información.

En el contexto de aplicaciones backend desarrolladas con Spring Boot, esta división adopta una forma adaptada:

- Capa de tráfico (*controllers*): gestiona las solicitudes y las respuestas HTTP.
- Capa de servicios: implementa la lógica de negocio y coordina las operaciones.
- Capa de acceso a datos (*repositories*): administra la persistencia y el acceso a bases de datos.

Esta estructura se complementa con el uso de entidades, DTO y un sistema robusto de manejo de excepciones para garantizar respuestas claras y consistentes.

Anotaciones en Spring Boot

Las anotaciones en Java son metadatos que indican cómo debe comportarse una clase o un método. En Spring Boot, resultan esenciales para implementar la arquitectura de tres capas.

Anotaciones comunes por capa:

- Capa de tráfico: @RestController, @RequestMapping, @GetMapping, @PostMapping.
- Capa de servicios: @Service, @Autowired.
- Capa de datos: @Entity, @Id, @GeneratedValue, @Repository.

Estas anotaciones simplifican el código y hacen más declarativa la estructura de la aplicación.

Buenas prácticas en la arquitectura de tres capas

- Separar responsabilidades claramente: evitar que la lógica de negocio se mezcle en los controladores.
- Usar DTO en las respuestas: proteger las entidades y reducir acoplamiento.
- Centralizar manejo de excepciones: garantizar mensajes claros y homogéneos.
- Aplicar Streams y Optional: mejorar la expresividad y la robustez del código.
- Mantener bajo acoplamiento: cada capa debe depender solo de la inmediatamente inferior.

¿Cómo se componen cada capa?

1. Capa de tráfico (*Controllers*)

La capa de tráfico actúa como punto de entrada a la aplicación. Se encarga de recibir las solicitudes HTTP enviadas por los clientes (navegadores, aplicaciones móviles, otros servicios) y de devolver la respuesta adecuada.

En Spring Boot, esta capa se implementa mediante clases anotadas con @RestController y métodos anotados con @GetMapping, @PostMapping, entre otros.

Un aspecto clave es el uso de la clase ResponseEntity, que permite devolver no solo los datos, sino también el código de estado HTTP (200, 404, etc.) y cabeceras adicionales, lo que aporta precisión en la comunicación con el cliente.

El siguiente ejemplo muestra un controller que recibe una solicitud GET /usuarios/{id}, delega la lógica al servicio (UsuarioService) y devuelve el resultado en un objeto UsuarioDTO.

Ejemplo:

```
@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    public UsuarioController(UsuarioService usuarioService) {
        this.usuarioService = usuarioService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<UsuarioDTO> obtenerUsuario(@PathVariable Long id) {
        UsuarioDTO usuario = usuarioService.buscarPorId(id);
        return ResponseEntity.ok(usuario);
    }
}
```

En este caso, el UsuarioController no contiene lógica de negocio: simplemente recibe la solicitud, obtiene el id del usuario y delega la operación en UsuarioService. El servicio se encarga de buscar los datos y devolver un UsuarioDTO.

Esto demuestra la idea central de la arquitectura de tres capas:

- La capa de tráfico se limita a comunicación HTTP.
- La lógica de negocio vive en la capa de servicios.
- La persistencia queda en la capa de datos.

De esta forma, el código es más claro, fácil de mantener y alineado con las buenas prácticas de desarrollo backend.

2. Capa de servicios

La capa de servicios constituye el núcleo de la aplicación. Aquí se definen las reglas de negocio y los procesos que gobiernan el funcionamiento del sistema. Su función es procesar los datos recibidos, ejecutar operaciones de negocio y coordinar la interacción con la capa de acceso a datos.

En Spring Boot, las clases de esta capa se anotan con @Service. Además, se recomienda aprovechar herramientas como Streams y Optional, que permiten trabajar con colecciones y valores potencialmente nulos de forma más clara y segura.

A continuación, se muestra un ejemplo de un servicio que busca un usuario por su id. Si el usuario existe, transforma la entidad en un DTO; si no, lanza una excepción controlada.

Ejemplo con Streams y Optional:

```java
@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    Qodo Gen: Options | Test this method
    public UsuarioDTO buscarPorId(Long id) {
        return usuarioRepository.findById(id)
                .map(usuario -> new UsuarioDTO(usuario.getNombre(), usuario.getEmail()))
                .orElseThrow(() -> new RecursoNoEncontradoException("Usuario no encontrado"));
    }
}
```

En este fragmento se observa cómo:

- El método findById devuelve un Optional, evitando errores por valores nulos.
- El método map transforma directamente la entidad en un UsuarioDTO.
- La excepción RecursoNoEncontradoException asegura una respuesta clara cuando no se encuentra el recurso.

Este patrón demuestra que la capa de servicios concentra la lógica de negocio y mantiene a los controladores libres de código innecesario, siguiendo las buenas prácticas de la arquitectura de tres capas.

Inyección de dependencias en la capa de servicios

Una de las ventajas más importantes de Spring Boot es que permite desacoplar las capas a través de la inyección de dependencias. Esto significa que el framework se encarga de crear y proveer los objetos que necesita cada clase, evitando que un Controller tenga que instanciar manualmente un Service, o que un Service cree un Repository.

Este mecanismo no solo reduce el acoplamiento, sino que también facilita el mantenimiento, la escalabilidad y el testing de la aplicación.

En Spring Boot existen dos formas principales de aplicar este patrón.

Ejemplo 1: Inyección con @Autowired en atributos

```java
@Service
public class UsuarioService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    Qodo Gen: Options | Test this method
    public UsuarioDTO buscarPorId(Long id) {
        return usuarioRepository.findById(id)
                .map(usuario -> new UsuarioDTO(usuario.getNombre(), usuario.getEmail()))
                .orElseThrow(() -> new RecursoNoEncontradoException("Usuario no encontrado"));
    }
}
```

Ventajas	Desventajas
Muy sencillo de implementar.	Menos explícito: la dependencia no se ve directamente en el constructor
Útil para introducir el concepto en las primeras prácticas.	Más difícil de testear, ya que no es tan simple inyectar mocks.

Ejemplo 2: Inyección por constructor

```java
@Service
public class UsuarioService {

    private final UsuarioRepository usuarioRepository;

    public UsuarioService(UsuarioRepository usuarioRepository) {
        this.usuarioRepository = usuarioRepository;
    }

    Qodo Gen: Options | Test this method
    public UsuarioDTO buscarPorId(Long id) {
        return usuarioRepository.findById(id)
                .map(usuario -> new UsuarioDTO(usuario.getNombre(), usuario.getEmail()))
                .orElseThrow(() -> new RecursoNoEncontradoException("Usuario no encontrado"));
    }
}
```

Ventajas	Desventajas
Más explícito: la dependencia es obligatoria y queda clara en el constructor.	Implica escribir un poco más de código que con @Autowired.
Facilita el testing: permite inyectar mocks o implementaciones personalizadas fácilmente.	
Es la práctica más utilizada en proyectos modernos con Spring.	

Tanto la inyección con @Autowired como la inyección por constructor cumplen la misma función: indicar a Spring Boot que debe proporcionar el objeto necesario. La diferencia está en el estilo de escritura y en el nivel de claridad que se busca en el código.

3. Capa de acceso a datos

La capa de acceso a datos se encarga de interactuar directamente con la base de datos. En Spring Boot suele implementarse con Spring Data JPA, que facilita la definición de repositorios mediante interfaces.

Las entidades (@Entity) representan las tablas de la base de datos y se gestionan mediante repositorios (@Repository).

Ejemplo:

```
@Entity
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column
    private String description;
}
```

```
@Repository
public interface CategoryRepository extends JpaRepository<Category, Long>{



}
```

De esta forma, la lógica de persistencia queda desacoplada y encapsulada en repositorios, mientras que las demás capas solo interactúan con los métodos expuestos por ellos.

Entidades, DTO y manejo de excepciones

Entidades

Las entidades son objetos que representan tablas en la base de datos. Son fundamentales en la capa de datos, pero no deberían exponerse directamente en la capa de tráfico.

DTO (Data Transfer Objects)

Los DTO son objetos diseñados para transportar datos entre capas. Permiten evitar el acoplamiento directo entre las entidades y las respuestas HTTP, aumentando la seguridad y la flexibilidad.

Ejemplo de DTO:

```
@Data
public class CategoryRequest {
    private Long id;
    private String description;
}
```

Lombok

La anotación @Data es de Lombok, y ahorra muchísimo código repetitivo al generar automáticamente:

- Getters
- Setters
- equals() y hashCode()
- toString()
- Un constructor requerido para los campos final

Ejemplo sin Lombok

Con solo @Data, ya se tienen todos los getters, los setters y los métodos útiles generados automáticamente.

```
@Entity
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String email;

    Qodo Gen: Options | Test this method
    // Getters y setters manuales
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

```java
@Entity
@Data
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private String email;
}
```

Excepciones y manejo de errores

Un sistema robusto de manejo de errores mejora la comunicación con el cliente. Spring Boot permite mapear excepciones a códigos de estado HTTP utilizando @ResponseStatus.

Ejemplo:

```java
@ResponseStatus(code = HttpStatus.BAD_REQUEST, reason = "La categoría que se intenta agregar esta duplicada")
public class CategoryDuplicateException extends Exception {

}
```

Así, cuando no se encuentra un recurso, la aplicación responde automáticamente con un 404 Not Found, proporcionando claridad al consumidor de la API.

Conclusión

La arquitectura de tres capas es la base de muchos sistemas backend modernos porque promueve orden, claridad y escalabilidad. Al dividir el sistema en tráfico, servicios y acceso a datos, se facilita la mantenibilidad y la evolución del software.

El uso de herramientas como ResponseEntity, Streams, Optional, DTOs y anotaciones específicas de Spring Boot refuerza este enfoque, alineando las buenas prácticas académicas con los estándares de la industria.

Comprender y aplicar esta arquitectura permitirá a los estudiantes construir proyectos robustos y preparados para escenarios más avanzados como microservicios o arquitecturas distribuidas.

Bibliografía

- SPRING.IO. *Spring Boot documentation.* [en línea]. [s.f.]. Disponible en: https://spring.io/projects/spring-boot
- ORACLE. *Java SE Documentation.* [en línea]. [s.f.]. Disponible en: https://docs.oracle.com/javase/
- BAELDUNG. *Guide to Spring Boot Annotations.* [en línea]. [s.f.]. Disponible en: https://www.baeldung.com/spring-boot-annotations

Referencia: CUELLO, Gisele. *Arquitectura de tres capas.* Buenos Aires: Universidad Argentina de la Empresa (UADE), 2025. Artículo no publicado.